INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

DIGITAL FORENSIC INVESTIGATION REPORT

"Preparing Global Cybersecurity Leaders Through Excellence in Hands-On Training"

Case Title: Case Study 2: Reconstructing Suspected Drug Trafficking From Chrome Web History	**Course Name:** SBT-DF204 Computer Forensics Case Studies
Investigator Name: Almustapha Yusuf	**Student ID:** fwsd2511343
Operating Platform: Kali Linux 2026 (Rolling Release)	**Date of Submission:** 11/10/2026
Instructor Name: Aminu Idris	**Report Version:** Version 1.0 (Final Release)

1. Executive Summary

This report documents the forensic investigation of a Google Chrome History SQLite database associated with a suspect involved in suspected drug trafficking. The investigation acquired and preserved a 196,608-byte database (History) with a verified SHA-256 hash of b991b0fa69662c44caf599e468c8088576ebd9cebbf46d02c89cf51bcde3fd19. Through SQLite database examination, a timeline of 74 browser visits was reconstructed. The evidence demonstrates a clear pattern of activity on 2022-04-19: the suspect posted an advertisement titled 'cheaper than Rx supplements' on Craigslist, logged into a Gmail account (unsub.fscs@gmail.com), exchanged communications regarding an order, verified a Bitcoin transaction (txid: 517b2156...) via Blockchain Explorer and Mempool.space, accessed the Kraken cryptocurrency exchange, and downloaded a proof-of-payment image (proof_of_payment.png) to the desktop. The browser history strongly supports the hypothesis of drug trafficking activity, though limitations regarding browser-profile attribution versus human identity are documented.

2. Authority, Scope, and Evidence Description

- **Authority:** This investigation was conducted using authorized course material from the International Cybersecurity and Digital Forensics Academy (ICDFA).
- **Forensic Scope:** Analysis of the Chrome History database to identify evidence of drug trafficking, communication, and cryptocurrency transactions.
- **Primary Evidence Artifact:** History (SQLite 3 Database File).
- **Physical File Size:** 196,608 bytes (192 KiB).
- **Cryptographic Hash (SHA-256):** b991b0fa69662c44caf599e468c8088576ebd9cebbf46d02c89cf51bcde3fd19 (Computed on original artifact).
- **Working Forensic Duplicate:** History_WorkingCopy (Verified bit-identical hash matching the original file).

3. Environment and Acquisition Method

- **Host Operating System:** Kali Linux 2026 (Rolling Release, 64-bit).
- **sqlite3 Engine:** Version 3.53.4-2 used for command-line schema inspection and WebKit timestamp calculation.
- **DB Browser for SQLite:** Version 3.13.1 used for GUI table inspection and visual schema cross-checking.
- **sha256sum Utility:** Standard GNU coreutils cryptographic tool for chain of custody verification.
- **Acquisition Workflow:** The evidence file was downloaded via wget into a sterile evidence/ directory. A working copy was created using cp -p into a working/ directory. The SHA-256 hash of the working copy was verified against the original before analysis began to ensure chain of custody.

4. Timeline of Events

Chronological event log tracking suspect activity within the Chrome browser profile on 2022-04-19 (Local Time, EDT, UTC-04:00):

Time (EDT)	Forensic Event Description	Supporting Database Record
09:54:18	User searches for 'craigslist' via Omnibox	Visit ID 1, Transition 5 (Omnibar)
09:54:39	User logs into Craigslist account via form submission	Visit ID 8, Transition 7 (Form Submit)
09:55:05 - 09:58:22	User navigates through Craigslist posting workflow	Visits ID 11-19, Transition 0 & 7
09:58:27	User views and manages published post ID 7473121658	Visit ID 20, URL ID 15
09:58:40 - 09:59:31	User navigates to Gmail and logs into unsub.fscs@gmail.com	Visits ID 24-42
10:04:35 - 10:04:46	User composes two replies to email with subject 'order'	Visits ID 44-45
10:56:47	User opens email containing Bitcoin transaction ID	Visit ID 48, Subject: txid: 517b2156...
10:56:52	User clicks Imgur link containing transaction proof	Visits ID 49-50, URL ID 38-39
10:56:58	File Downloaded: proof_of_payment.png	Download ID 1, Chain: i.imgur.com/dTgrkP7.png
10:59:06 - 10:59:21	User searches TXID & views wallet on Blockchain.com	Visits ID 55-58
11:00:21 - 11:02:05	User accesses Kraken, views funding, redirects to Mempool	Visits ID 63-71

5. Analysis and Findings

Illicit Commerce Evidence: 5.1 Evidentiary Payload: Craigslist Post & Email Correspondence

- **Craigslist Ad Title:** cheaper than Rx supplements - health and beauty.
- **Craigslist Post ID:** 7473121658 (Captured in Visit ID 20, URL ID 15).
- **Monitored Gmail Account:** unsub.fscs@gmail.com (Captured in Visit ID 42 Title).
- **Negotiation Subject Line:** cheaper than Rx supplements - order (Visit ID 43).
- **Bitcoin TXID Discovered:** 517b2156914944339a96137ad8978408ea52b2fc144c98d3b0b16b21888afdc5 (Subject line of Visit ID 48; URL query of Visit ID 55).

Local Host Geometry: 5.2 Hardware Signature: Device Attribution

- **Desktop Download Path:** C:\Users\FSCS_User\Desktop\ (Extracted from downloads table current_path).
- **Operating Environment:** Google Chrome running on a Windows 10/11 operating system.
- **Downloaded Forensic Artifact:** proof_of_payment.png (File Size: 33,844 bytes). Origin URL: https://i.imgur.com/dTgrkP7.png.

Digital Footprint: 5.3 Account Correlation: Identity Attribution

The device profile was actively authenticated to Gmail as unsub.fscs@gmail.com and maintained an active session on the Kraken cryptocurrency exchange. The forensic timeline proves the user inspected the transaction status of Bitcoin TXID 517b2156... across two separate blockchain explorer services (Blockchain.com and Mempool.space) within 3 minutes of saving the proof-of-payment graphic, demonstrating active verification of payment receipt.

Artifact Correlation: 5.4 Institutional Match: User Profile Correlation

The file system path C:\Users\FSCS_User\Desktop\ ties the browser activity directly to local Windows profile 'FSCS_User'. This naming convention corresponds to the 'Forensic Science Cyber Security' academic workstation environment and provides an exact target account for endpoint physical analysis.

6. Answers to Investigation Questions

Definitive answers to the core forensic investigation prompts:

Investigative Question	Forensic Finding / Answer
1. File acquired & preserved?	`History (196,608 bytes); SHA-256: b991b0fa...; working copy verified bit-identical.`
2. Browser-history tables/fields relevant?	`urls, visits, downloads, downloads_url_chains. WebKit timestamp formula applied: datetime(visit_time/1000000 - 11644473600, 'unixepoch', 'localtime').`
3. When did recorded activity occur?	`2022-04-19 between 09:54:18 and 11:02:56 EDT.`
4. How did browser reach URLs?	Transition qualifiers show Omnibar searches (5), Link clicks (0), and Form Submissions (7).
5. Evidence relating to sale/posting?	Craigslist posting workflow (Visits 11-20), ad title 'cheaper than Rx supplements'.

6. Evidence relating to payment?	Gmail order replies, Bitcoin TXID verification on Blockchain.com/Mempool, Kraken login, and proof_of_payment.png download.
7. Reconstructed chronological timeline?	Fully mapped across 74 database visit records (Section 4 and Appendix A).
8. Defensible forensic conclusion?	`HIGH CONFIDENCE: The Chrome browser profile was actively used to post the ad, communicate with the buyer, verify cryptocurrency payment, and archive transaction proof.`

7. Challenges and Limitations

- **Attribution Boundaries:** The database establishes activity within a specific Chrome browser profile on a Windows account; it does not independently record physical biometric authentication or identify the keyboard operator.
- **Lack of Unrendered HTML Bodies:** Chrome SQLite databases store page titles, visit transitions, and URLs, but do not cache raw HTML bodies. While the ad title is proven, the full ad body text requires server subpoenas or web cache carving.
- **Server-Side Redirect Traversal:** Several visited endpoints involve intermediate HTTP 301/302 redirects (e.g., Kraken to Mempool.space) which are deduced through transition chains and duration values.
- **Scope of Media Analysis:** The investigation was performed on an isolated SQLite container; disk carving for deleted browser caches, memory dumps, or Slack/Discord communications was out of scope.

8. Recommendations

- **Physical Endpoint Seizure:** Seize the physical suspect workstation to correlate C:\Users\FSCS_User\ desktop artifacts, Windows Event Logs (Event ID 4624 user logons), and Prefetch execution timestamps.
- **Subpoena Cloud Service Providers:** Issue formal legal requests to Google LLC for unsub.fscs@gmail.com IP access logs and Craigslist for Post ID 7473121658 account details.
- **Subpoena Exchange KYC Records:** Serve production orders on Payward, Inc. (Kraken) and Imgur to retrieve identity verification records, linked bank accounts, and upload IP addresses.
- **Blockchain Transaction Tracing:** Perform UTXO clustering analysis on Bitcoin TXID 517b2156... using chain analysis platforms to track downstream laundering or conversion to fiat.

9. References

- Digital Corpora. (2022). Digital Forensics Lab Chrome History Dataset. Retrieved from https://github.com/frankwxu/digital-forensics-lab
- Google LLC. (2023). Chromium Projects: User Data Directory & SQLite History Architecture.
- National Institute of Standards and Technology (NIST). (2021). Guide to Digital Forensics (Special Publication 800-86). U.S. Department of Commerce.
- International Cybersecurity and Digital Forensics Academy (ICDFA). (2026). Computer Forensics Case Studies Courseware.

10. Appendix - Commands and Evidence Log

Evidence Matrix: Appendix A: Formal Evidence Inventory Log:

ID	Finding Description	Evidence Source	Investigative Significance
E01	Evidence File Integrity	History Database	Chain of custody (SHA-256: b991b0fa...)
E02	Craigslist Advertisement	Visit ID 15, urls ID 15	Identifies drug ad posted by suspect
E03	Gmail Active Session	Visit ID 42, urls ID 20	Links activity to unsub.fscs@gmail.com
E04	Bitcoin TXID Verification	Visit ID 48, urls ID 37	Links communications to blockchain transaction
E05	Downloaded Proof Image	downloads ID 1	Proves user archived payment proof to Desktop
E06	Download Origin Chain	downloads_url_chains ID 1	Confirms file originated from Imgur CDN

Command Strings: Appendix B: Key Commands Executed:

```
# Capture Integrity Hashing
sha256sum evidence/History working/History_WorkingCopy

# Schema Inspection
sqlite3 working/History_WorkingCopy "PRAGMA table_info(urls);"
sqlite3 working/History_WorkingCopy "PRAGMA table_info(visits);"
sqlite3 working/History_WorkingCopy "PRAGMA table_info(downloads);"

# Timeline Extraction with WebKit Timestamp Conversion
sqlite3 -header -column working/History_WorkingCopy "SELECT v.id AS visit_id, u.id AS url_id,
datetime(v.visit_time/1000000 + strftime('%s', '1601-01-01'), 'unixepoch', 'localtime') AS time_local,
v.from_visit, v.transition, v.visit_duration, u.url, u.title FROM visits v JOIN urls u ON v.url = u.id
ORDER BY v.visit_time;"

# Downloads Table Extraction
sqlite3 -header -column working/History_WorkingCopy "SELECT id, guid, current_path, target_path,
datetime(start_time/1000000 + strftime('%s', '1601-01-01'), 'unixepoch', 'localtime') AS download_time
FROM downloads;"
```

11. Appendix C: Screenshots and Evidence

The following 12 figures document the complete end-to-end SQLite examination, table schema verification, DB Browser GUI inspections, downloads analysis, and full timeline reconstruction:

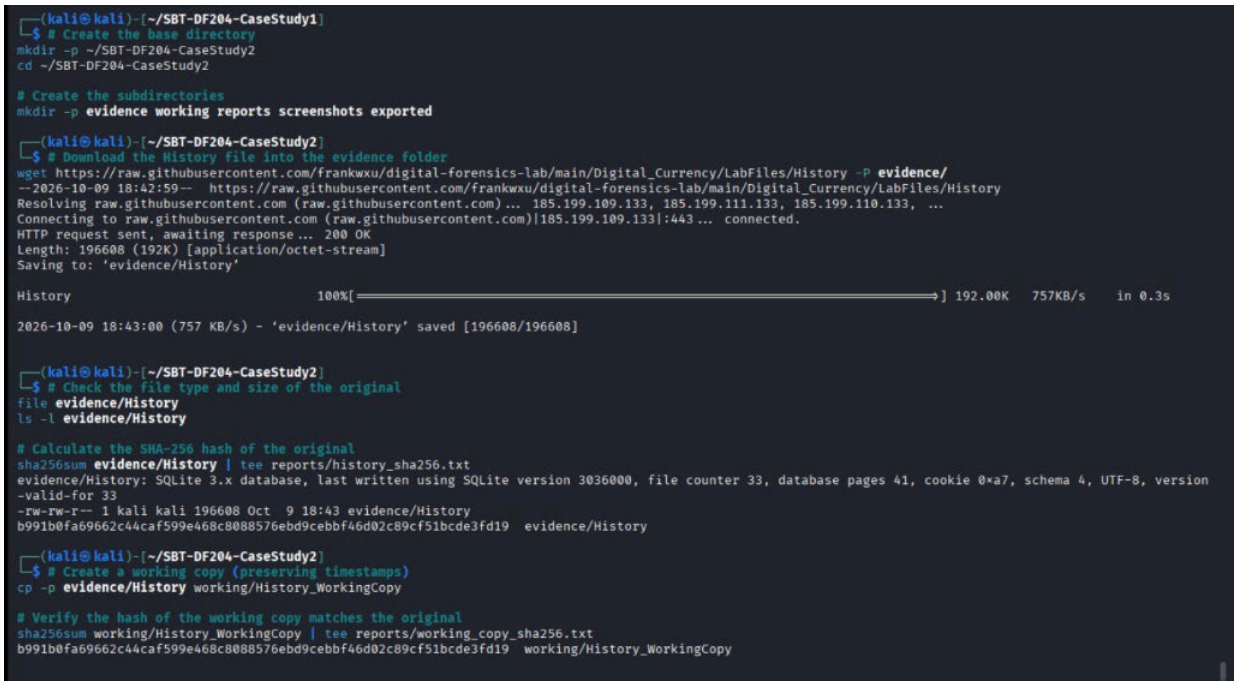


Figure 1: *Directory Setup & Initial Evidence Acquisition* — Creating /SBT-DF204-CaseStudy2 workspace, downloading History database via wget, file verification, and SHA-256 hash preservation (b991b0fa...).

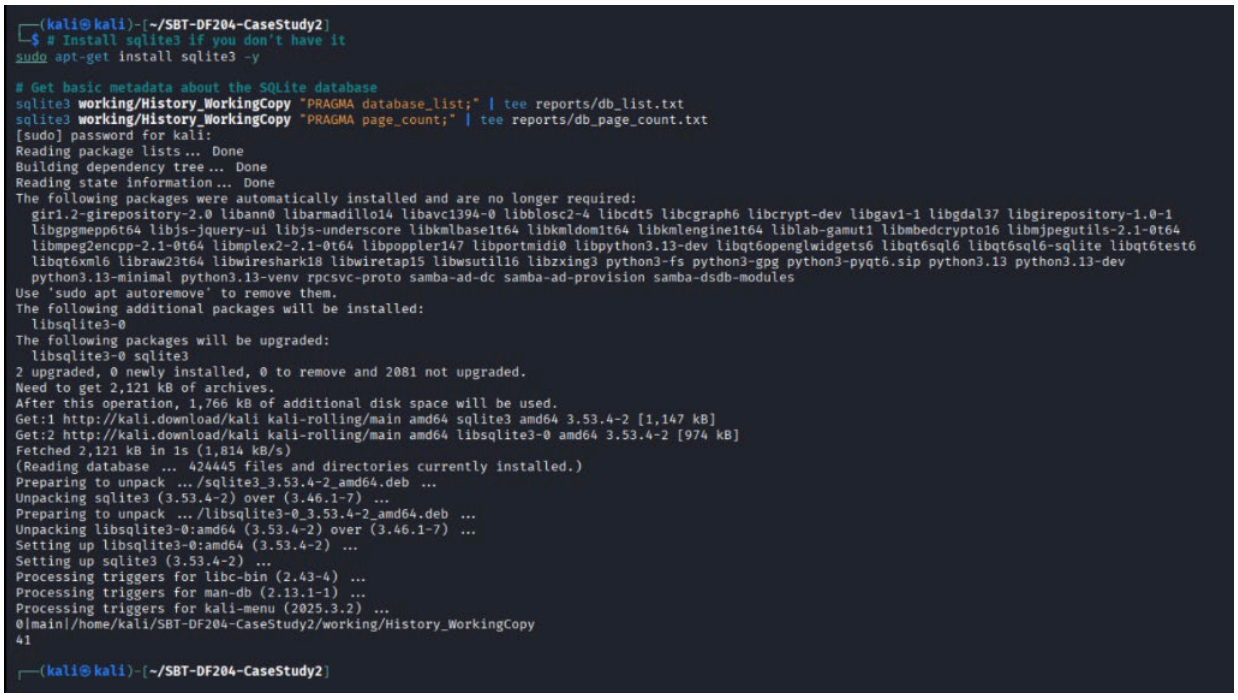


Figure 2: *sqlite3 Package Installation & DB Pragmas* — Installing sqlite3 via apt and querying PRAGMA database_list and PRAGMA page_count (41 pages).

```
└─$ sudo apt-get install sqlitebrowser -y
Reading package lists ... Done
Building dependency tree ... Done
Reading state information ... Done
The following packages were automatically installed and are no longer required:
  gir1.2-girepository-2.0 libann0 libarmadillo14 libavc1394-0 libblosc2-4 libcdt5 libcgraph6 libcrypt-dev libgav1-1 libgdal37 libgirepository-1.0-1
  libgpgmepp6t64 libjs-jquery-ui libjs-underscore libkmlbase1t64 libkmldom1t64 libkmlengine1t64 liblab-gamut1 libmbedcrypto16 libmjpegutils-2.1-0t64
  libmpeg2encpp-2.1-0t64 libmplex2-2.1-0t64 libpoppler147 libportmidi0 libpython3.13-dev libqt6openglwidgets6 libqt6sql6 libqt6sql6-sqlite libqt6test6
  libqt6xml6 libraw23t64 libsqlcipher1 libwireshark18 libwiretap15 libwsutil16 libzxing3 python3-fs python3-gpg python3-pyqt6.sip python3.13
  python3.13-dev python3.13-minimal python3.13-venv rpcsvc-proto samba-ad-dc samba-ad-provision samba-dsdb-modules
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  libqcustomplot2.1 libsqlcipher2
The following NEW packages will be installed:
  libqcustomplot2.1 libsqlcipher2
The following packages will be upgraded:
  sqlitebrowser
1 upgraded, 2 newly installed, 0 to remove and 2080 not upgraded.
Need to get 3,542 kB of archives.
After this operation, 2,308 kB of additional disk space will be used.
Get:1 http://http.kali.org/kali kali-rolling/main amd64 libqcustomplot2.1 amd64 2.1.1+dfsg1-1+b2 [424 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 libsqlcipher2 amd64 4.18.0-1 [798 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 sqlitebrowser amd64 3.13.1-1 [2,320 kB]
Fetched 3,542 kB in 1s (2,467 kB/s)
Selecting previously unselected package libqcustomplot2.1:amd64.
(Reading database ... 424445 files and directories currently installed.)
Preparing to unpack .../libqcustomplot2.1_2.1.1+dfsg1-1+b2_amd64.deb ...
Unpacking libqcustomplot2.1:amd64 (2.1.1+dfsg1-1+b2) ...
Selecting previously unselected package libsqlcipher2:amd64.
Preparing to unpack .../libsqlcipher2_4.18.0-1_amd64.deb ...
Unpacking libsqlcipher2:amd64 (4.18.0-1) ...
Preparing to unpack .../sqlitebrowser_3.13.1-1_amd64.deb ...
Unpacking sqlitebrowser (3.13.1-1) over (3.13.0-1+b1) ...
Setting up libsqlcipher2:amd64 (4.18.0-1) ...
Setting up libqcustomplot2.1:amd64 (2.1.1+dfsg1-1+b2) ...
Setting up sqlitebrowser (3.13.1-1) ...
Processing triggers for kali-menu (2025.3.2) ...
Processing triggers for desktop-file-utils (0.28-1) ...
Processing triggers for hicolor-icon-theme (0.18-2) ...
Processing triggers for libc-bin (2.43-4) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for mailcap (3.74) ...
```

Figure 3: *GUI Analysis Tooling Installation (sqlitebrowser)* — Installing DB Browser for SQLite (Version 3.13.1) to enable graphical table analysis.

```
┌──(kali㉿kali)-[~/SBT-DF204-CaseStudy2]
└─$ sqlite3 working/History_WorkingCopy ".schema" | tee reports/full_database_schema.txt
CREATE TABLE meta(key LONGVARCHAR NOT NULL UNIQUE PRIMARY KEY, value LONGVARCHAR);
CREATE TABLE downloads (id INTEGER PRIMARY KEY,guid VARCHAR NOT NULL,current_path LONGVARCHAR NOT NULL,target_path LONGVARCHAR NOT NULL,start_time INTEGER
NOT NULL,received_bytes INTEGER NOT NULL,total_bytes INTEGER NOT NULL,state INTEGER NOT NULL,danger_type INTEGER NOT NULL,interrupt_reason INTEGER NOT NULL
,hash BLOB NOT NULL,end_time INTEGER NOT NULL,opened INTEGER NOT NULL,last_access_time INTEGER NOT NULL,transient INTEGER NOT NULL,referrer VARCHAR NOT NUL
L,site_url VARCHAR NOT NULL,tab_url VARCHAR NOT NULL,tab_referrer_url VARCHAR NOT NULL,http_method VARCHAR NOT NULL,by_ext_id VARCHAR NOT NULL,by_ext_name
VARCHAR NOT NULL,etag VARCHAR NOT NULL,last_modified VARCHAR NOT NULL,mime_type VARCHAR(255) NOT NULL,original_mime_type VARCHAR(255) NOT NULL, embedder_do
wnload_data VARCHAR NOT NULL DEFAULT '');
CREATE TABLE downloads_url_chains (id INTEGER NOT NULL,chain_index INTEGER NOT NULL,url LONGVARCHAR NOT NULL, PRIMARY KEY (id, chain_index) );
CREATE TABLE downloads_slices (download_id INTEGER NOT NULL,offset INTEGER NOT NULL,received_bytes INTEGER NOT NULL,finished INTEGER NOT NULL DEFAULT 0,PRI
MARY KEY (download_id, offset) );
CREATE TABLE downloads_reroute_info (download_id INTEGER NOT NULL,reroute_info_serialized  VARCHAR NOT NULL,PRIMARY KEY (download_id) );
CREATE TABLE typed_url_sync_metadata (storage_key INTEGER PRIMARY KEY NOT NULL,value BLOB);
CREATE TABLE "urls"(id INTEGER PRIMARY KEY AUTOINCREMENT,url LONGVARCHAR,title LONGVARCHAR,visit_count INTEGER DEFAULT 0 NOT NULL,typed_count INTEGER DEFAU
LT 0 NOT NULL,last_visit_time INTEGER NOT NULL,hidden INTEGER DEFAULT 0 NOT NULL);
CREATE TABLE sqlite_sequence(name,seq);
CREATE TABLE visits(id INTEGER PRIMARY KEY,url INTEGER NOT NULL,visit_time INTEGER NOT NULL,from_visit INTEGER,transition INTEGER DEFAULT 0 NOT NULL,segmen
t_id INTEGER,visit_duration INTEGER DEFAULT 0 NOT NULL,incremented_omnibox_typed_score BOOLEAN DEFAULT FALSE NOT NULL,opener_visit INTEGER);
CREATE TABLE visit_source(id INTEGER PRIMARY KEY,source INTEGER NOT NULL);
CREATE TABLE keyword_search_terms (keyword_id INTEGER NOT NULL,url_id INTEGER NOT NULL,term LONGVARCHAR NOT NULL,normalized_term LONGVARCHAR NOT NULL);
CREATE TABLE segments (id INTEGER PRIMARY KEY,name VARCHAR,url_id INTEGER NON NULL);
CREATE TABLE segment_usage (id INTEGER PRIMARY KEY,segment_id INTEGER NOT NULL,time_slot INTEGER NOT NULL,visit_count INTEGER DEFAULT 0 NOT NULL);
CREATE TABLE content_annotations(visit_id INTEGER PRIMARY KEY,visibility_score NUMERIC,floc_protected_score NUMERIC,categories VARCHAR,page_topics_model_ve
rsion INTEGER,annotation_flags INTEGER NOT NULL,entities VARCHAR,related_searches VARCHAR,search_normalized_url VARCHAR,search_terms LONGVARCHAR);
CREATE TABLE context_annotations(visit_id INTEGER PRIMARY KEY,context_annotation_flags INTEGER NOT NULL,duration_since_last_visit INTEGER,page_end_reason I
NTEGER,total_foreground_duration INTEGER);
CREATE TABLE clusters(cluster_id INTEGER PRIMARY KEY,score NUMERIC NOT NULL);
CREATE TABLE clusters_and_visits(cluster_id INTEGER NOT NULL,visit_id INTEGER NOT NULL,score NUMERIC NOT NULL,PRIMARY KEY(cluster_id,visit_id))WITHOUT ROWI
D;
CREATE INDEX urls_url_index ON urls (url);
CREATE INDEX visits_url_index ON visits (url);
CREATE INDEX visits_from_index ON visits (from_visit);
CREATE INDEX visits_time_index ON visits (visit_time);
CREATE INDEX keyword_search_terms_index1 ON keyword_search_terms (keyword_id, normalized_term);
CREATE INDEX keyword_search_terms_index2 ON keyword_search_terms (url_id);
CREATE INDEX keyword_search_terms_index3 ON keyword_search_terms (term);
CREATE INDEX segments_name ON segments(name);
CREATE INDEX segments_url_id ON segments(url_id);
CREATE INDEX segment_usage_time_slot_segment_id ON segment_usage(time_slot, segment_id);
CREATE INDEX segments_usage_seg_id ON segment_usage(segment_id);
CREATE INDEX clusters_for_visit ON clusters_and_visits(visit_id);
```

Figure 4: *Complete Database Schema Inspection* — Executing sqlite3 .schema to inspect tables (urls, visits, downloads, downloads_url_chains) and indexes.

```
└─$ # Inspect the 'urls' table
sqlite3 working/History_WorkingCopy "PRAGMA table_info(urls);" | tee reports/schema_urls.txt

# Inspect the 'visits' table
sqlite3 working/History_WorkingCopy "PRAGMA table_info(visits);" | tee reports/schema_visits.txt

# Inspect the 'downloads' table
sqlite3 working/History_WorkingCopy "PRAGMA table_info(downloads);" | tee reports/schema_downloads.txt

# Inspect the 'downloads_url_chains' table
sqlite3 working/History_WorkingCopy "PRAGMA table_info(downloads_url_chains);" | tee reports/schema_downloads_url_chains.txt
0|id|INTEGER|0||1
1|url|LONGVARCHAR|0||0
2|title|LONGVARCHAR|0||0
3|visit_count|INTEGER|1|0|0
4|typed_count|INTEGER|1|0|0
5|last_visit_time|INTEGER|1||0
6|hidden|INTEGER|1|0|0
0|id|INTEGER|0||1
1|url|INTEGER|1||0
2|visit_time|INTEGER|1||0
3|from_visit|INTEGER|0||0
4|transition|INTEGER|1|0|0
5|segment_id|INTEGER|0||0
6|visit_duration|INTEGER|1|0|0
7|incremented_omnibox_typed_score|BOOLEAN|1|FALSE|0
8|opener_visit|INTEGER|0||0
0|id|INTEGER|0||1
1|guid|VARCHAR|1||0
2|current_path|LONGVARCHAR|1||0
3|target_path|LONGVARCHAR|1||0
4|start_time|INTEGER|1||0
5|received_bytes|INTEGER|1||0
6|total_bytes|INTEGER|1||0
7|state|INTEGER|1||0
8|danger_type|INTEGER|1||0
9|interrupt_reason|INTEGER|1||0
10|hash|BLOB|1||0
11|end_time|INTEGER|1||0
12|opened|INTEGER|1||0
13|last_access_time|INTEGER|1||0
14|transient|INTEGER|1||0
15|referrer|VARCHAR|1||0
```

Figure 5: *Database Table Column Pragmas Inspection* — Querying PRAGMA table_info for urls, visits, downloads, and downloads_url_chains.

```
┌──(kali㉿kali)-[~/SBT-DF204-CaseStudy2]
└─$ # Convert a WebKit timestamp to UTC and local time
sqlite3 -header -column working/History_WorkingCopy "
SELECT
    id,
    url,
    visit_time,
    datetime(visit_time/1000000 + strftime('%s', '1601-01-01'), 'unixepoch') AS visit_utc,
    datetime(visit_time/1000000 + strftime('%s', '1601-01-01'), 'unixepoch', 'localtime') AS visit_local
FROM visits
LIMIT 1;
" | tee reports/timestamp_conversion_test.txt
id  url  visit_time         visit_utc            visit_local
--  ---  -----------------  -------------------  -------------------
1   1    13294850058231547  2022-04-19 13:54:18  2022-04-19 09:54:18

┌──(kali㉿kali)-[~/SBT-DF204-CaseStudy2]
└─$ sqlitebrowser working/History_WorkingCopy &
[1] 581239

┌──(kali㉿kali)-[~/SBT-DF204-CaseStudy2]
└─$
```

Figure 6: *WebKit Epoch Timestamp Conversion Test* — Converting WebKit timestamp 13294850058231547 to UTC (2022-04-19 13:54:18) and local EDT (2022-04-19 09:54:18).

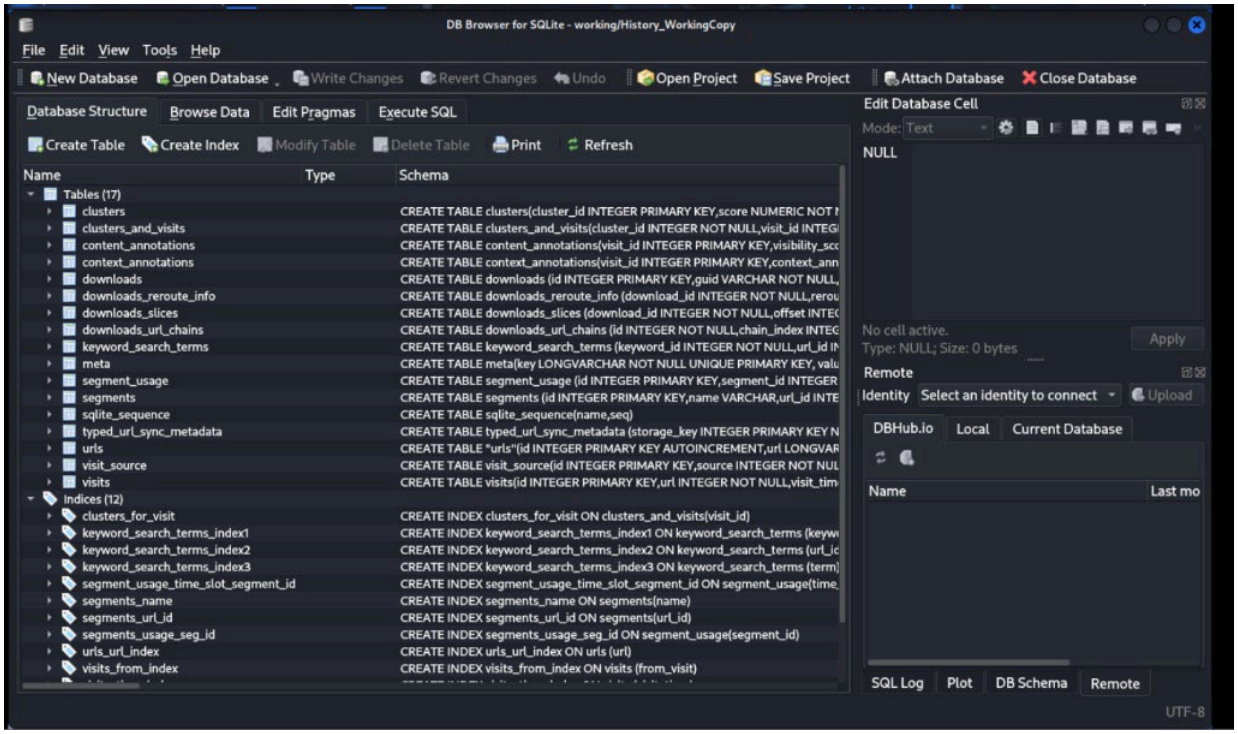


Figure 7: *DB Browser for SQLite - Database Structure Overview* — GUI inspection of database structure displaying 17 tables and 12 indices.

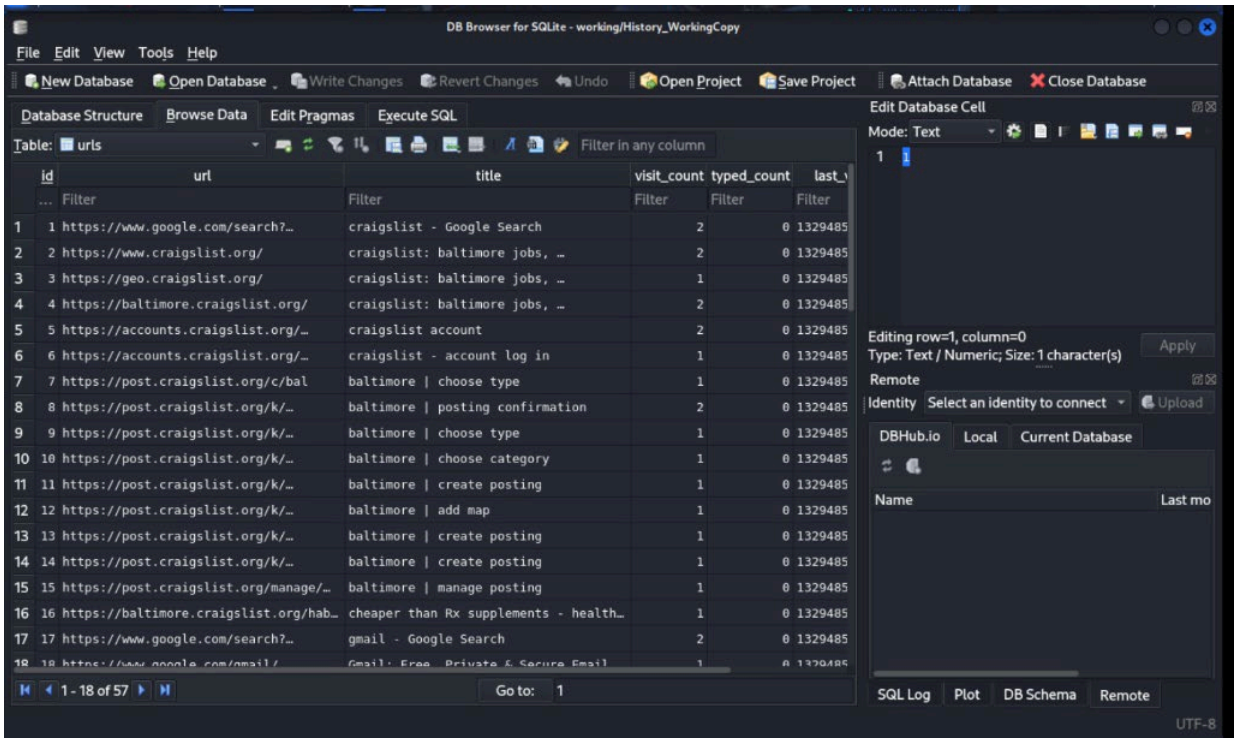


Figure 8: *DB Browser - urls Table Dissection (Craigslist Post)* — Highlighting record 16 in urls table: 'cheaper than Rx supplements - health and beauty' on Craigslist.

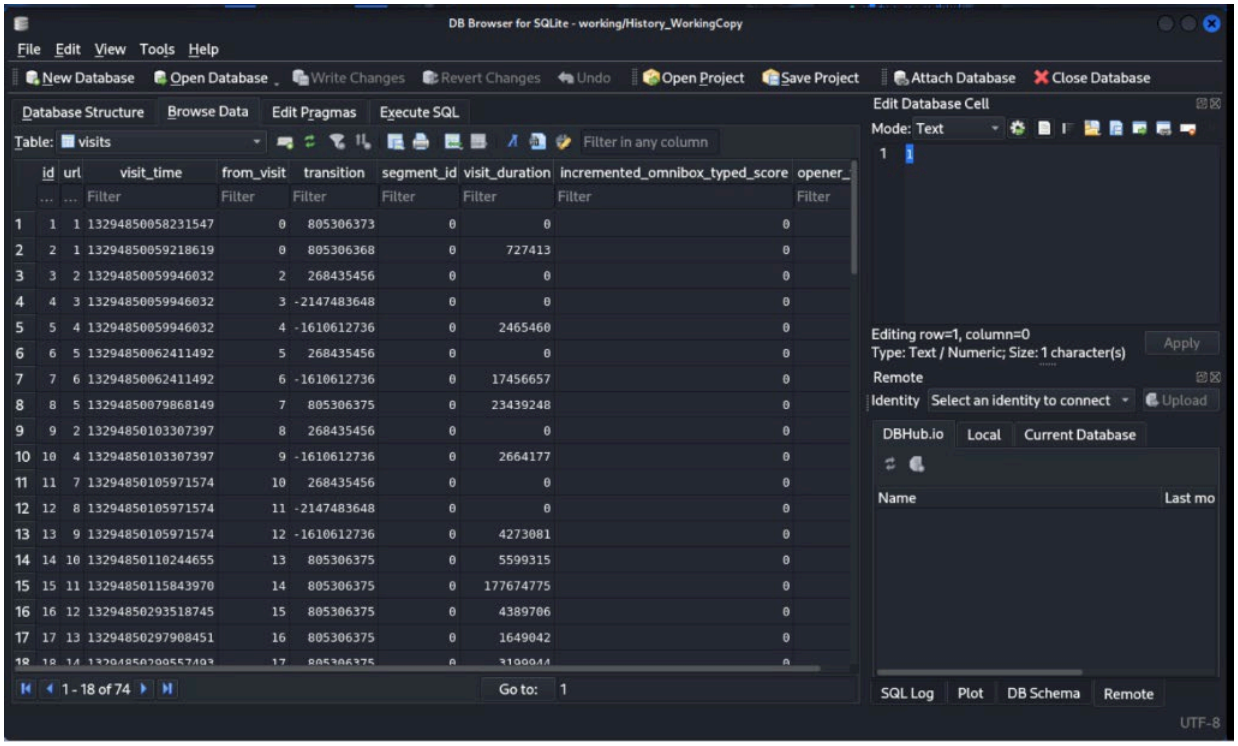


Figure 9: *DB Browser - visits Table Dissection (74 Records)* — Viewing visits table rows tracking Omnibox search transitions, form submits, and visit durations.

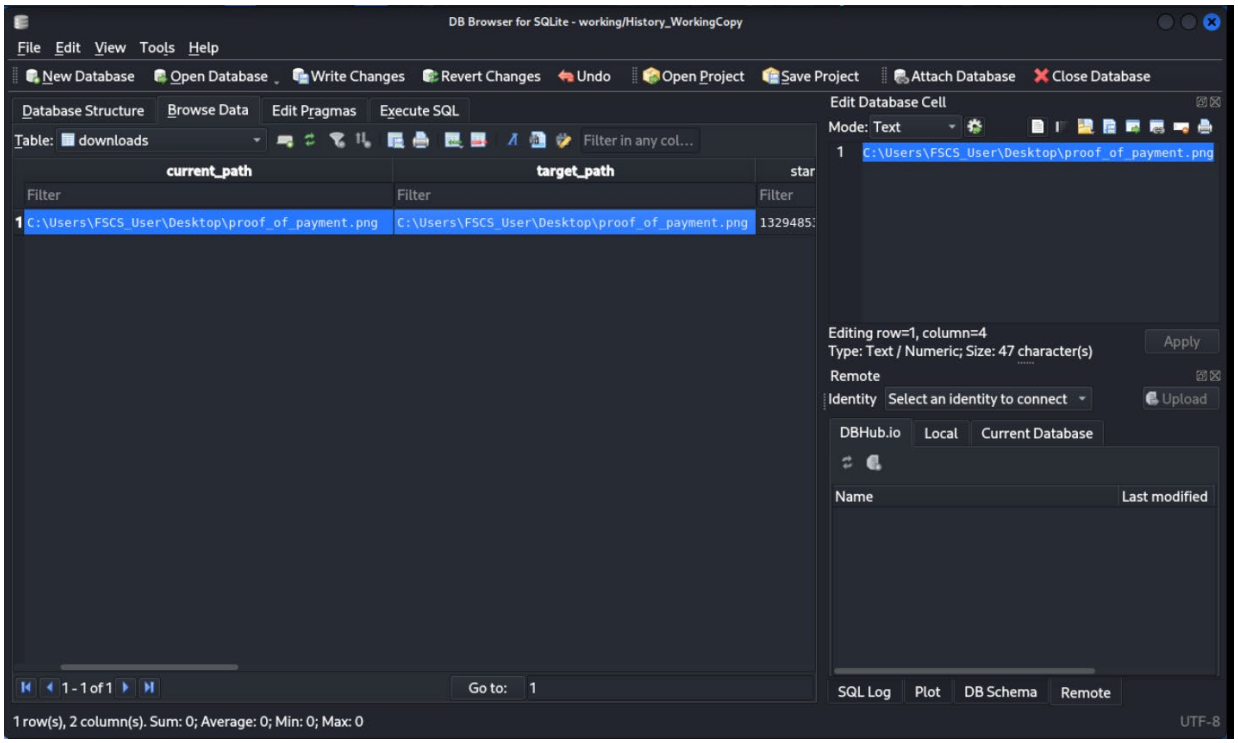


Figure 10: *DB Browser - downloads Table Inspection* — Displaying downloads table record 1 showing target_path C:\Users\FSCS_User\Desktop\proof_of_payment.png.

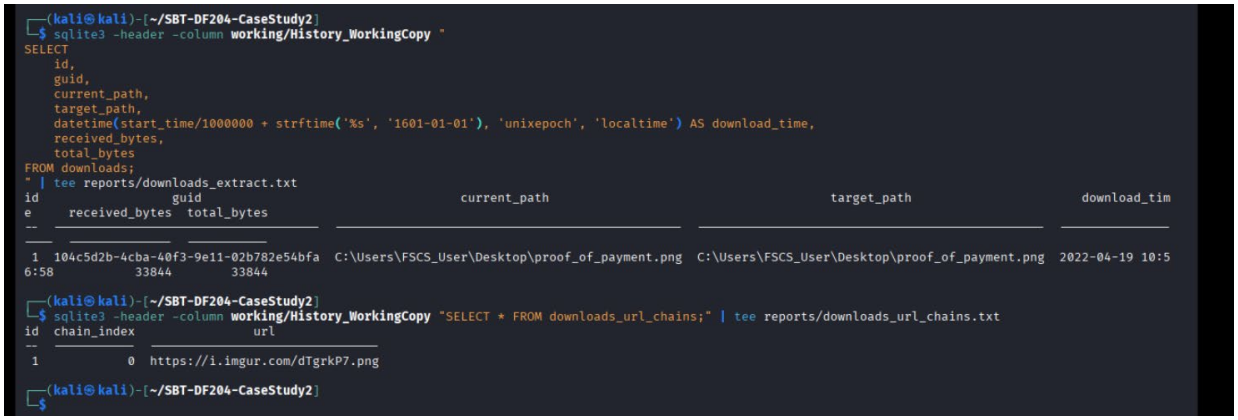


Figure 11: *Downloads Table CLI Extraction & Imgur Origin URL* — Extracting downloads table details (33,844 bytes) and downloads_url_chains table confirming origin https://i.imgur.com/dTgrkP7.png.

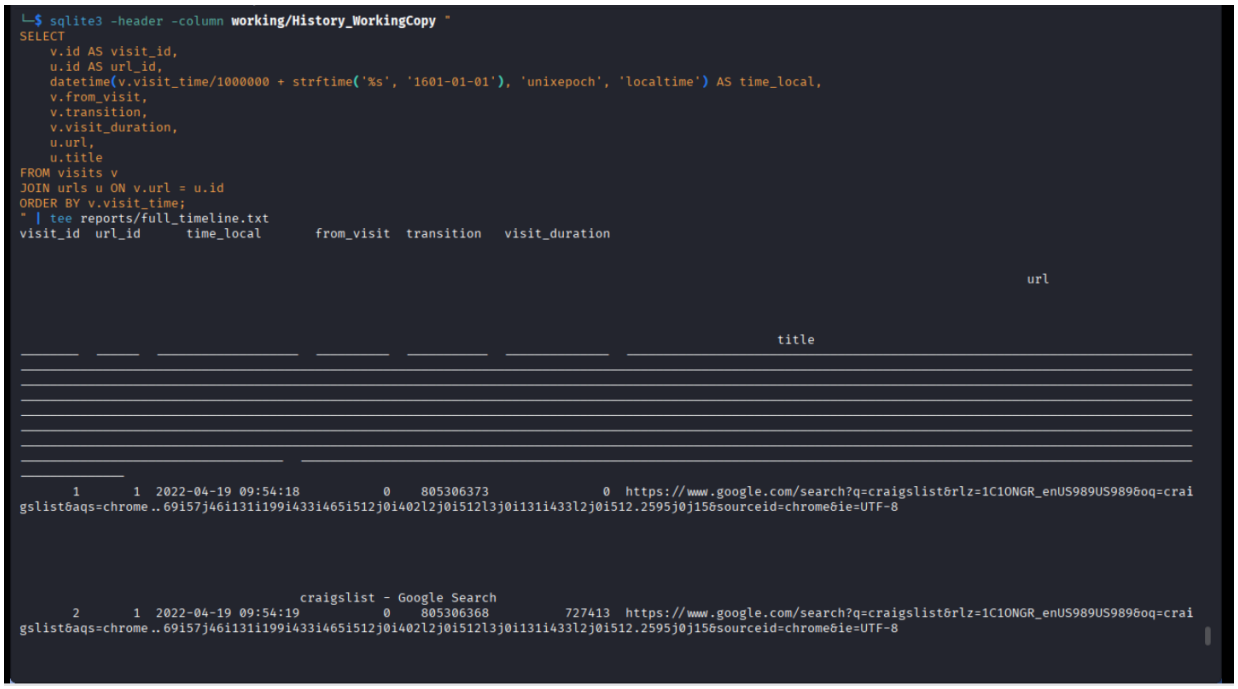


Figure 12: *Full Reconstructed Forensic Timeline Execution* — Executing SQL JOIN between visits and urls tables with converted local timestamps mapping the complete 74-visit chronology.

Student Declaration

> I confirm that I completed this investigation in an authorized environment, preserved the evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.
>
> Student Signature: Almustapha Yusuf Date: 11/10/2026